

S P E C T R A V I D E O

U S E R S G R O U P -- Wgtn

Newsletter Number 7 -- January/February 1985

Welcome to 1985

The first meeting for the year is on Monday ,February 25th at 7.30 in the Staff Common Room , Wellington Clinical School . Mein Street Newtown.

There will be a demonstration of the new double sided disk drives and hopefully a demo of the Coleco Games adapter and Turbo Pascal for CP/M buffs .

At 7.00 there will be a 20 minute tutorial on BASIC , covering the use of formatting output using TAB , SPC and PRINT USING.

The following 2 meetings are on March 18 and April 22 , same place and time.

Wgtn SV Users Group

Secretary / Newsletter Editor

Don Stanley Ph 896379 (hm) , 746906 (work)

Treasurer

Colin Chin Ph 325656 (hm)

Committee members

Ross Wakelin

Mike Dalliessi 848416

Dennis Timmons 837200

Darryl Alison 85522 (Paraparaumu) (hm) 399510 (wk)

New members Welcome to A N Bannatyne , J Blemford , Russell Clover , B M Collis , Brad Dixon , Ian Galbraith , Tony Guilford , Ron Hughes , Karl Langston , David Lindsay , Michael Parker , David Wilcock , H M Williamson , John Walsh , Bill Fergusson , N Church , Tony Otter , G Burdan , Mr Gooder , G Silcock , Mr Tucker , Mr Carter , D Holtslag , Mr Conroy , Kevin Wronski , R L Neilson , R Brathwaite

Membership cards will be sent out shortly .

Number of Members = 106

Newsletter Contributions

If you are submitting programs to go in the newsletter they must satisfy the following conditions or they will NOT be printed.

- 1 ... be typed/printed on A4 paper with about 5 lines left blank at top and bottom of page
- 2 ... typed with a dark ribbon OR printed using emphasized print mode
- 3 ... include a description of what the program does , typed on A4 paper with example input
- 4 ... In the case of programs above 20 lines in length they should be sent on cassette , in this case there is no need for the printout as I can merge them into the newsletter. Enclose a stamped addressed envelope for return of cassette
- 5 ... line numbers should be continuous -- do a RENUM before printing the programs
- 6 ... PLEASE MAKE SURE THEY WORK . and would readers please note that I am NOT responsible for programs sent by people which do not work .
- 7 ... In the case of screen dumps from 3d plotters please indicate the function used and try to get the print as dark as possible -- not always easy i know .

All other newsletter contributions should be sent to me at P.O Box 7057 , Wellington South. They may miss being printed for a month if sent to any other box or given to anyone else. Articles should follow 1 and 2 above.

Deadline for Newsletter articles is the 30/31st of each month. Please do not send articles direct to the printer.

Discounts for members .. there seems to be some confusion about these , especially among members who do not attend meetings. All discounts offered by Moonshine Computers , Einstein Scientific and Microstyle Computers do NOT include hardware. Discounts may be offered on such peripherals as floppy disks, cassettes , books , furniture, printers , monitors but NOT on SpectraVideo hardware. Individual dealers may not offer all the goods above at discounted rates , inquire from the shops just what discounts are available to club members. Note that to obtain a discount on any

goods you must produce your current membership card.

Recalling accidentally NEWed programs. Our semi-regular scribe Deane Whitmore has come with the following for recalling a program which you just did an accidental NEW on. This assumes that line 10 of your program is REM .

type in direct mode POKE &H8001,&H07 : POKE &H8002,&H80

Whats going on here is that the position of the second line is being poked into memory. When a NEW is done , the first 2 bytes of user ram , &h8001 and &h8002 are filled with nulls , generally these two bytes contain the start location of the next line of basic. The routine will work with a REM in line 1 only , because ANY other basic command would make line one longer , therefore the position of the second line would be in a different , generally unknown place,

This will restore your program in memory , but the NEW command also destroys any jump table addresses in memory so the program cannot be RUN , only listed. Save the program to tape or disk and then reload it -- hey presto away you go.

The major limitation here is obviously the need to have a REM in the first line. If you can get round this limitation how about letting me know for the newsletter.

A note on REM. I was experimenting with various odd things recently and stumbled on why the ' uses so much ram when used as a program REM. In case you didn't know , the REM command uses one byte of memory , using ' uses 3 bytes. This is because internally a ' is converted to the following .

:REM <space> your REMark

In other words ,10 ' demo program is equivalent to entering 10 :REM demo program. Since the word REM uses 1 byte of memory (it is tokenised as &h8F) and the : and space a byte each , you use 3 bytes whenever ' is used. Thus to conserve space in a program use REM , not ' , for remarks starting at column 1.

We all know that MASS TAEBS machine code games are superb , and how frustrating they are are to load. If you own a disk drive you can copy them to disk as follows ..

type BLOAD "SASA"
 BSAVE "1:SASA",&h8B00,&h8B00+16*1024,&h8B00

and you now have a copy of SASA , or whatever game , on disk. To

MICROSTYLE COMPUTERS LIMITED
198 JACKSON STREET, P.O. BOX 38-479, PETONE. TELEPHONE 686-963

HOURS	9.00AM - 5.00PM	MONDAY TO THURSDAY
	9.00AM - 9.00PM	FRIDAYS
	9.00AM - 1.00PM	SATURDAYS

IN ADDITION TO THE FULL RANGE OF SVI SPECTRAVIDEO PRODUCTS WE ALSO CARRY THE FULL RANGE OF "PROGRANZ" SOFTWARE AND PRODUCTS, AS WELL AS THE FULL RANGE OF BUSINESS PROGRAMS FOR THE SPECTRAVIDEO FROM RGW SYSTEMS, THE STOCK CONTROL/ACCOUNTS PROGRAM FROM SL MICROSYSTEMS AND THE CARPE SERIES OF STOCK CONTROL/INVOICING, DEBTOR'S, CREDITORS AND GENERAL LEDGER FROM CARPE OFFICE SYSTEMS (N.Z.) LTD.

PROGRANZ SOFTWARE TITLES. #18 ON TAPE #23 ON DISK

- P101 QUICKDRAW:-A SCREEN DRAWING PROGRAM SUITS CHILDREN 2YEARS AND OVER.
- P103 KIWI COUNTRY - NORTH ISLAND :-A WELL WRITTEN PROGRAM WHICH SHOULD BE A MUST IF YOU HAVE SCHOOL AGED CHILDREN
- P106 TYPMASTER:- EDUCATIONAL PROGRAM AND TYPING TUTOR
- P107 WORLD ATLAS:-GENERAL KNOWLEDGE, EDUCATIONAL PROGRAM WHICH INCLUDES A GRAPHIC WORLD MAP(TAPE VERSION #15)
- P108 WORLD ATLAS EX (REQUIRES EXPANDED 318):-EXPANDED VERSION OF WORLD ATLAS(P107) MORE COUNTRIES, LEADERS, CLUES AND CAPITALS AS WELL AS MOVEMENT OF SUN TIME, MONTH, YEAR SEASONS.
- P121 SPECTRONIC CALCULATOR:-THIS VERSATILE PROGRAM INCLUDES A METRIC CONVERSION MODE AND A LISTED CALCULATION MODE AS WELL AS THE NORMAL FEATURES OF A CALCULATOR
- P122 SKILLMAN:-EDUCATIONAL GAME PROGRAM
- P123 BATS ALIVE:-GAME PROGRAM (TAPE VERSION #15)
- P124 T.V. NIGHTMARE:-GAME PROGRAM "LONG TERM PLAYABILITY"
- P141 MATHS 1 :- BEGINERS MATHS WITH SIMPLE GRAPHICS
- P142 MATHS 2 :- CARRIES ON FROM MATHS ONE
- P151 FLIGHT SIMULATOR(REQUIRES EXPANDED 318):- PILOT A 747 JUMBO JET AROUND NEW ZEALAND AND AUSTRALIA (AVAILABLE SOON)

R.G.W SYSTEMS BUSINESS SOFTWARE(DISK DRIVE REQUIRED).

FINANCIAL REPORTING SYSTEM(SINGLE DISK VERSION)	\$139.00
FINANCIAL REPORTING SYSTEM(TWIN DISK VERSION)	\$155.00
JOB COSTING SYSTEM	\$65.00
STOCK CONTROL SYSTEM(80 COL CARD REQUIRED)	\$80.00

load it type `BLOAD"1:SASA",r`. Do not insert the ,R when doing the above cassette load. Once the game is loaded you can switch off the expander to stop the drive turning , it will not affect the game.

There is an extension to the play command which is often overlooked , this tells you which of the sound queues are active.

`X=PLAY(0) : Y=PLAY(1) : Z=PLAY(2) : W=PLAY(3)`

The value of X will be 0 if none of channels A,B,C is active or -1 if ANY channel is active. Y is zero if channel A is not active , otherwise -1 . Z is zero if channel B is not active , otherwise -1. W is 0 if channel C is not active , otherwise -1.

Here is a useful ROM routine , especially for machine code applications. The routine at &H48E1 checks the screen mode. If the mode is text-mode an illegal function call error is generated. Screen1 leaves the value 0 in the A-register , while Screen2 leaves the value 1 in the A-register. Thus this routine is a check that a graphics function is being called from a graphics screen mode.

This routine always generates an error if in text mode. The alternative which doesn't produce an error is to look at the contents of location &hFE3A. This location returns 0 , 1 or 2 depending on the current screen mode.

B A S I C N O T E S

How basic stores programs.

What you see when you LIST a program is an ASCII listing of your program . If Basic were to store your program in memory exactly as you see it , the memory overheads would be enormous. Consequently , a technique known as tokenising is used.

Tokenising means to take all the basic reserved words , and rather than store the ASCII word , replace it with a 1 or 2 byte code instead. Thus the word AND , which is 3 bytes long , is stored as the 1 bytes code &HFB. A full list of these tokens is presented further on , thanks to Brian Wilson for these.

Along with the tokenised words , Basic stores text and linenumbers , as well as pointers to the next line. The text is such things as strings in your program , names of variables and so on. Lines are stored as follows.

The first two bytes contain the location in memory where the next

line of basic code starts. These two bytes are reversed. The next two bytes contain the current line number. Two bytes are needed to allow line numbers up to 65535, but only 1 byte is used if the line number is below 256. Again the bytes are reversed. Then your line of tokenised basic code is stored, terminated with a null byte.

Heres an example of storage of basic. The program is as follows.

```
400 deffna(x)=cos(x)
500 print fna(1)
```

Note first that all programs start at location &h8001 unless you force them to do otherwise.

The internal storage looks like this.

```
loc &h8001 +1 +2 +3 +4 +5 +6 +7 +8 +9 +10 +11 +12 +13 +14 +15 +16
byte      12 80 90 01 97 de 41 28 58 29 f1 ff 8c 28 58 29 00
```

```
loc &h8012 +1 +2 +3 +4 +5 +6 +7 +8 +9 +10 +11 +12 +13 +14 +15 +16
byte      1d 80 f4 01 91 de 41 28 12 29 00 00 00 00 00 00 00
```

Locations &h8001 & &h8002 are the location of the next line of basic, in this case 1280, or on reversing the bytes, &h8012. Then 2 bytes containing the current line number, 9001 or on reversing, &h0190 which in decimal is 400, our first linenumber. The line of basic code starts next. Location &h8001+4 contains a 97, the list of tokens reveals this as the token for a DEF, and the next byte, DE, is the token for FN. The next few bytes are untokenised, their ASCII equivalent is a(x) (ie 41,28,58,29). The next byte, F1 is the token for an =, while ff 8c is the token for cos. Then the untokenised code is (x), followed by a null to end the line.

Ok, the start of the next line of basic code is at &h8012. The two bytes at &h8012,&h8013 contain the pointer to the next line, in this case &h801d. Then the f401 (really 01f4) is decimal 500, our new line number. The next byte, 91, is the token for PRINT, then we have the token for FN followed by a(1). The next null is the line terminator, and the null following that is at &h801d, this null at the next line number indicates the end of the program.

The following list contains all the reserved words, plus the tokens, plus the memory location where the machine code routine corresponding to the command starts.

KeyWord	Token	Entry Point	KeyWord	Token	Entry Point
AUTO	A9	11F5	AND	F8	
ABS	FF 06	55B1	ATN	FF 0E	5139
ASC	FF 15	6B10	.ATTR\$	E9	
BSAVE	CE	7624	BLOAD	CD	7684
BEEP	C0	40BE	BIN\$	FF 1D	6904

CLICK	C8	31AF
COPY	D6	34BF
CLEAR	92	67A6

CLOSE	B4	7375
CONT	99	671B
CLOAD	9B	1EAA

KeyWord	Token	Entry Point	KeyWord	Token	Entry Point
CSAVE	9A	1E15	CSRLIN	EB	
CINT	FF 1E	56B5	CSNG	FF 1F	56DD
CDBL	FF 20	5765	CVI	FF 2B	732B
CVS	FF 29	732E	CVD	FF 2A	7331
COS	FF 0C	50B8	CHR\$	FF 16	6B20
CIRCLE	BC	2652	COLOR	BD	4552
CLS	9F	3777	CMD	D7	34C4
DELETE	AB	1C60	DATA	B4	109B
DIM	B6	6061	DEFSTR	AB	0F5C
DEFINT	AC	0F5F	DEFSNG	AD	0F62
DEFDBL	AE	0F65	DSKO\$	D1	34A6
DEF	97	188A	DSKI\$	EA	
DSKF	FF 26	34C9	DRAW	BE	29DA
DIAL	D0	79C2	ELSE	A1	109D
END	B1	66CF	ERASE	A5	676E
ERROR	A6	11EA	ERL	E1	
ERR	E2		EXP	FF 0B	526B
EOF	FF 2B	74B0	EQV	FB	
FOR	B2	0D65	FIELD	B1	72CD
FILES	B7	73B2	FN	DE	
FRE	FF 0F	6CF7	FIX	FF 21	57E9
FPOS	FF 27	74C6	GOTO	B9	102B
GO TO	B9	102B	GOSUB	BD	0FF6
GET	B2	2FB4	HEX\$	FF 1B	68FF
INPUT	B5	13D2	IF	BB	1225
INSTR	E5		INT	FF 05	57F8
INP	FF 10	1A37	IMP	FC	
INKEY\$	EC		IML	D5	34BA
KILL	D4	34B5	KEY	C7	3120
LPRINT	9D	125D	LLIST	9E	1AB3
LPOS	FF 1C	1834	LET	BB	01C0
LOCATE	D8	2FD1	LINE	AF	1374
LOAD	B5	7121	LSET	BB	722B
LIST	93	1AB8	LFILES	BB	73AD
LOG	FF 0A	5197	LOC	FF 2C	74B4
LEN	FF 12	6B04	LEFT\$	FF 01	6B66
LOF	FF 2D	749A	MOTOR	CC	2BE5
MERGE	B6	7122	MOD	FD	
MKI\$	FF 2E	7312	MKS\$	FF 2F	7315
MKD\$	FF 30	7318	MID\$	FF 03	6B9F
MON	CB	7B44	MAX	CA	7CBA
MDM	CF	3036	NEXT	B3	6B21
NAME	D3	34B0	NEW	94	6556
NOT	E0		OPEN	B0	70B0
OUT	9C	1A4C	ON	95	1124
OR	F9		OCT\$	FF 1A	6BFA
OFF	EB		PRINT	91	1265
PUT	B3	2FB1	POKE	9B	1CAD
POS	FF 11	1839	PEEK	FF 17	1CA6

PSET C2 232D
POINT ED
PDL FF 24 3280

PRESET C3 2328
PAINT BF 24FC
PAD FF 25 32BD

KeyWord	Token	Entry Point	KeyWord	Token	Entry Point
PLAY	C1	2C24	RETURN	8E	1061
READ	B7	1405	RUN	8A	0FE2
RESTORE	8C	66AE	REM	8F	109D
RESUME	A7	119D	RSET	B9	7227
RIGHT\$	FF 02	6B96	RND	FF 08	5300
RENUM	AA	1CF0	SCREEN	C5	459A
SPRITE	EE		SWITCH	C9	337F
STOP	90	66CB	SWAP	A4	6735
SET	D2	34AB	SAVE	BA	7167
SPC(	DF		STEP	DC	
SGN	FF 04	55C6	SQR	FF 07	5222
SIN	FF 09	50D1	STR\$	FF 13	6909
STRING\$	E3		SPACE\$	FF 19	6B4D
SOUND	C4	2BFD	STICK	FF 22	3206
STRIG	FF 23	3263	THEN	DA	
TRON	A2	672F	TROFF	A3	6730
TAB(	DB		TO	D9	
TIME	EF		TAN	FF 0D	5120
USING	E4		USR	DD	
VAL	FF 14	6BC0	VARPTR	E7	
VPOKE	C6	46DB	VPEEK	FF 18	46F2
WIDTH	A0	1A6C	WAIT	96	1A52
XOR	FA		+	F3	
-	F4		*	F5	
/	F6		\	FE	
=	F1		^	F7	
>	F0		<	F2	

Some of these have no entry point listed. They are secondary keywords, which means that they follow another keyword (eg USING is meaningless on its own, but follows PRINT). These keywords have no jump table address, and consequently the entry points are hidden somewhere in your machines innards. If you find any of these entry points, please let me know.

Also the tokens for the functions beginning FF appear to need to have &h80 added to them.

Reminder - - -

If you have some small and seemingly insignificant problem or piece of knowledge please let me know. The whole point of the club is to exchange information and ideas to the benefit of all. Likewise if you discover some small routine or something useful (like how to stop the disk drives spinning) please let me know for inclusion in the newsletter, it is sure to be useful to someone. Any problems which are sent to me will be answered in

the newsletter , or if I can't find the answer I will print the query in the newsletter and hope someone else can answer it.

See #9 (8) Notes: issue 9 pgs

A query from an out of town member is how to input a function to a program instead of having to change line whatever when you want to use a function. I have come up with a routine here which is demonstrated in the following program to solve equations using the bisection method. The routine is fairly long , I have a detailed explanation of it following.

The subroutine beginning at line 400 is the function input routine and only other lines needed are lines 90 and 100 which do not have to have those numbers , the routine is vastly more efficient if these 3 lines are placed as near as possible to the top of the program (after any clear statements).

```
20 REM Bisection Method
40 CLEAR 350
50 WIDTH40
70 REM next chunk of code required for function input
90 GOSUB 400
100 REM!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
    !!!!!!!!!!!!!!!
120 REM function is entered --- now talk to user
140 LOCATE5,5:INPUT"Enter Start points "; S1,S2
150 LOCATE5,5:INPUT"Enter convergence epsilon "; EPS
160 IF SGN(FNA(S1))=SGN(FNA(S2)) THEN LOCATE5,20:PRINT"Start
    /End":GOSUB880:END
170 R1=100
180 R0 = FNA((S1+S2)/2)
190 IT = IT +1:IF IT > 20 THEN GOSUB 310:END
200 IF ABS(R0-R1)<EPS THEN GOSUB230:END
210 IF SGN(R0)=SGN(FNA(S1)) THEN S1=(S1+S2)/2 ELSE S2=(S1+S2)/2
220 R1=R0:GOTO 180
230 REM root found -- print information
260 CLS
270 LOCATE1,5:PRINT "Number of Iterations = ";IT
280 LOCATE1,7:PRINT USING"Final Root      = ###.####";
    (S1+S2)/2
290 LOCATE1,9:PRINT "Function was ";NF$
300 GOSUB880:RETURN
310 REM did not converge in 20 iterations -- stop
340 CLS
350 LOCATE1,5:PRINT"Limit of 20 Iterations Exceeded "
360 LOCATE1,7:PRINT"No further iterations allowed  "
370 LOCATE1,9:PRINTUSING"Last root estimate was ###.####";R0
380 LOCATE1,11:PRINTUSING"Previous estimate was ###.####";R1
390 GOSUB880:RETURN
400 REM input users function
430 FORI=&H001T00:&H8F AND PEEK(I+1)=&H21 THEN TP=I ELSE
440 CLS
450 LOCATE 5,7:PRINT"Enter your Function"
460 LOCATE 5,8:PRINT"as a function of X eg X*X or x*x"
```

```

470 LOCATE 5,9:PRINT"Up to 70 characters are acceptable "
480 LOCATE 5,12:INPUT A$:A$=" "+A$+" ":NF$=" 0 = "+A$
490 FOR I=1 TO LEN(A$)
500 J=ASC(MID$(A$,I,1))
510 IF J<123 AND J>96 THEN MID$(A$,I,1)=CHR$(J AND 95)
520 NEXT
530 POKE TP,&H97:POKE TP+1,&HDE:POKE TP+2,&65:POKE TP+3,&40
540 POKE TP+4,&88:POKE TP+5,&41:POKE TP+6,&HF1
560 REM tokenise things
580 B$="+":J=&HF3:GOSUB950
590 B$="-":J=&HF4:GOSUB950
600 B$="*":J=&HF5:GOSUB950
610 B$="/":J=&HF6:GOSUB950
620 B$="^":J=&HF7:GOSUB950
630 B$="\":J=&HFE:GOSUB950
640 B$=">":J=&HF0:GOSUB950
650 B$="=":J=&HF1:GOSUB950
660 B$="<":J=&HF2:GOSUB950
670 B$="ABS":J=&HB6:GOSUB1040
680 B$="ATN":J=&HB8:GOSUB1040
690 B$="COS":J=&HB0:GOSUB1040
700 B$="EXP":J=&HB2:GOSUB1040
710 B$="FIX":J=&HA1:GOSUB1040
720 B$="INT":J=&HB5:GOSUB1040
730 B$="LOG":J=&HBA:GOSUB1040
740 B$="RND":J=&HB8:GOSUB1040
750 GOSUB1090
760 B$="SGN":J=&HB4:GOSUB1040
770 B$="SQR":J=&HB7:GOSUB1040
780 B$="SIN":J=&HB9:GOSUB1040
790 B$="TAN":J=&HB0:GOSUB1040
810 REM now poke the function into memory
830 FOR I=1 TO LEN(A$)
840 POKE TP+6+I,ASC(MID$(A$,I,1))
850 NEXT
860 POKE TP+6+I,&H3A:POKE TP+7+I,&HBF:POKE TP+8+I,&HE6
870 RETURN
880 REM restore REM!!!! ... line
910 POKE TP,&HBF
920 TP=TP+1
930 IF PEEK(TP) <> &H21 THEN POKE TP,&H21 ELSE RETURN
940 GOTO 920
950 REM tokenise arithmetic operator or function
980 IF INSTR(A$,B$)=0 THEN 1000
990 MID$(A$,INSTR(A$,B$))=CHR$(J) : GOTO 980
1000 RETURN
1020 REM tokenise function
1040 IF INSTR(A$,B$)=0 THEN 1080
1050 I=INSTR(A$,B$)
1060 A$=LEFT$(A$,I-1)+CHR$(&HFF)+CHR$(J)+RIGHT$(A$,LEN(A$)-I-2)
1070 GOTO 1040
1080 RETURN
1090 REM tokenise mod
1120 IF INSTR(A$,"MOD")=0 THEN 1160
1130 J=INSTR(A$,"MOD")

```

```

1140 A$=LEFT$(A$,J-1)+CHR$(&HFD)+RIGHT$(A$,LEN(A$)-J-3)
1150 GOTO 1120
1160 RETURN

```

Ignoring the code which solves the equation (its only there to demonstrate the function input routine), whats happening is as follows.

Lines 90 and 100 are required, 90 to call the function input routine and 100 to set aside space for the function. Line 100 looks a bit odd, it sets aside space for the function by initially filling a block of memory with exclamation marks. (You could use any character which takes your fancy, but the REM is absolutely necessary).

As is fairly obvious the input routine uses the keyword tokens to store your function. The first line of the subroutine (420) searches through memory for the first occurrence of a line beginning REM! . (The token for REM is &h8f, ! has no tokens as anything following a REM is stored as ASCII text). This is why the routine is more efficient if the REM!!!! ... line occurs as near possible to the top (recall the program starts at &h8001). The location where this line occurs (called TP) is where we will eventually begin to store the function. If the REM!!! line is omitted the program just keeps on searching .. and searching ... and searching

The function is input and stored in two strings, A\$ and NF\$. A\$ is going to be altered a bit, so a copy of the ASCII string is kept in NF\$ in case you want to print it. The following FOR loop at lines 490 -- 520 just converts any text to uppercase. This ensures that, for instance cos and COS are the same thing, and means that we don't need separate processing for cos and COS.

The two lines of POKes are the beginning of the function definition. &h97 is poked into location TP, look thru the list of tokens and find that &h97 is DEF, the next byte &hDE is the token for FN. The byte going into location TP+2 can be virtually anything, it is the function name, in this case A, while the next 3 POKes poke (x) into memory. The final POKE pokes an = sign into location TP+6 (recall &hf1 is the token for =)

At this stage we have replaced REM!!!!!! with DEFFNA(X)= . Now the function which you typed in needs to be tokenised. The next 22 lines look for the operators and functions in B\$ (see why I made the input text uppercase now) and replace the ASCII text which you entered with the tokens. For instance if your function is x*cos(x), it tokenises to the string x chr\$(&hf5) chr\$(&hff) chr\$(&h8c) (x) . Thats what the final three subroutines do, they tokenise various functions and operators. I haven't got all of the functions and operators here, for instance none of the logical operators are here, so that if you typed in AND an error would eventually occur.

After the 22 tokenising lines comes the actual poking of the

function into memory. Since A\$ is now tokenised where it needs to be , just poke the values of A\$ into location TP+7 up to TP+LEN(A\$).

There are 3 more pokes to follow. They just replace the next 3 exclamation marks with :REM<space> which is equivalent to '. The reason I use a ' here instead of spacing out all remaining exclamation marks is because its quicker.

The function is now in line 100, since the subroutine was called from line 90 the next line processed is line 100 , the function definition.

The only other thing which needs explaining here is the routine at 880 , this undoes the whole process by restoring line 100 to REM!!!!!! Call it at the end of your program so that the program will still work next time (otherwise line 100 reads DEFFNA(X)= ... '!!!!!! and the search for REM! will fail).

If you want to check that the routine really works , leave out the call to routine 880 at the end , you will see DEFFNA(X)=

A further point to note is that any errors in your function input will be detected when you try to use the function , not at input time.

GAMES SCORES

Spectron	9700	Alexander Lindsay
Sector Alpha	666400	Gavin Knight
Kung Fu	15835	Angela Chin
S C Force	748600	Deirdre Chin
Telebunny	87810	Colin Chin
	26470	Don Stanley (+)
Omac	25700	Don Stanley (+)
Ninja	53400	Colin Chin
Tetra	256750	Colin Chin
Turboat	68390	Garth Thornton (*)
Sasa	45120	Michael Wright

(+) = started in normal level

(*) - started in arcade level

otherwise practice level or unknown

Magazine Sponsors

Moonshine Computers -- 68 Ghuznee Street , World Trade Center

Einstein Scientific -- 177 Willis Street , Wellington

MicroStyle Computers -- 198 Jackson Street , Petone

A D V E R T I S E M E N T

E I N S T E I N

S C I E N T I F I C

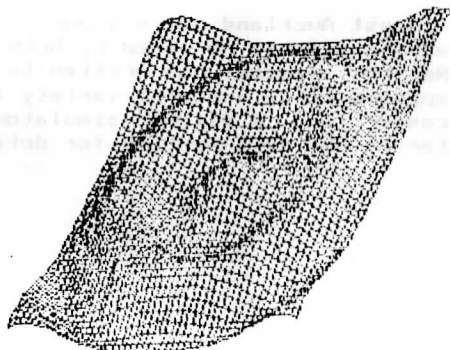
177 WILLIS STREET , Wellington , ph 851055

S P E C T R A V I D E O dealers

CURRENTLY AVAILABLE -- A LARGE STOCK OF
SPECTRAVIDEO 80 COLUMN CARDS , SINGLE
SLOT EXPANDERS , AND CENTRONICS PRINTER
INTERFACE CARTRIDGES.

The SV605b DOUBLED SIDED Disk Drives can
be ordered -- 10 days delivery time from
date of order. The SV605a Disk Drives are
in stock NOW .

Call Michael Pollard or Raju Bariani for more
details --- 177 Willis Street , Phone 851-055



Back Copies of Magazine

Number 1 is completely gone , I will repeat a couple of articles from it in future issues. I have very few copies of numbers 2,3 and 4 left , but 5 and 6 are Ok at present. Back copies are \$1.00 each.

Unfortunately I have still not heard anything back from the Australian Users Group , there is their latest newsletter here which will appear at the next meeting . As far as connectors go I am trying to get a local company to make some of these up , but no details available yet.

A new computer magazine is starting up shortly . Called Just Computers this magazine is licensed to reprint articles from BYTE and from POPULAR COMPUTER , which means that material from these magazines will appear here at the same time as in the States , not 2 or 3 months later. The magazine has a newsstand price of NZ\$2.95 and will contain in excess of 200 pages. If we have anyone who wishes to contribute articles about SpectraVideo at any time please send such to

Just Computers
P.O Box 22435
AUCKLAND

Ph 276-8781

A yearly subscription is available at a cost of \$29.00 . Send to the above address with details of name , address etc . There are two yearly and three yearly subscriptions available also at a cost of \$55 and \$79 respectively .

According to the latest Auckland users group newsletter ,there is additional software available released by both Alpine Computing and PrograNZ. Neither company has written to me about this , however there appears to be a useful variety including a disk based Text Processor and a flight simulator. See the last Auckland newsletter at the next meeting for details.

The Wairarapa Users Group is holding meetings on the first Wednesday of each month. They have had one meeting to date attended by 16 people. The club contact is

Iain Scott
61 Fergusson Street
Masterton

ph 81133

A new addition to the club library is the Waikato Users Group newsletter .

Harry Patterson from Nelson is particularly interested in graphics and would like to hear from anyone else who has the same interest. If you are interested in swapping ideas , programs etc with Harry please let me know and I will forward addresses rather than print them in the newsletter .

And a query from Harry about disassemblers. For anyone else in the dark about what a disassembler does and is here goes. The Basic operating system which sits in the background on your computer is responsible for making Basic work. Essentially it provides an interface (BASIC) between you and the Z80 Cpu . This interface is written in machine code which is stored internally as screeds of binary digits. There are many many routines in there which can be used by programmers if they can find and interpret them. Here comes the disassembler. It takes the machine code and converts it in assembly language mnemonics which are a slightly more human way of writing the code. With a bit of experience in machine code you can work out (SLOWLY) what the mc routines do and perhaps use them in your own programs. So basically a disassembler just converts memory images back into their corresponding assembler mnemonics. A word of warning though -- it is a rare disassembler which can tell when it is disassembling machine code and when it is disassembling ascii text.

If you want to get into machine code at all what we really need is an assembler , which takes the assembly language mnemonics and converts them to machine code. In my humble opinion this is the single outstanding fault with SpectraVideo -- no assembler makes it very difficult to write any decent sized machine code program.

Graphics Routine From Harry Paterson

```
10 print tab(14) "Color Mixer"
20 for t=1 to 600:next:cls
30 color15,1,1:default a-z
40 y=0:d=1:e=2:f=255
50 input "Enter First Color";a
60 input "Enter Second Color";b
70 screen 2
80 for x=d to f step e
90 line(x,y)-(x,y),a:nextx
100 for x=e to f step e
110 line(x,y)-(x,y),b:nextx
120 y=y+d
130 goto 80
```

```
10 print tab(14) "Color Mixer"
20 for t=1 to 600:next:cls
30 color15,1,1:default a-z
40 c=191:d=255:e=0:f=2:g=1
50 input "Enter First Color";a
60 input "Enter Second Color";b
70 screen 1:for y=f to c step f
80 line (e,y)-(d,y),a,bf:next y
90 for y=g to c step f
100 line(e,y)-(d,y),b,bf
110 next
120 goto 120
```

Screen Dump routine for MICROLINE 92 printer from John Matthews

```
1000 a$=input$(1)
1010 defint k,z,g : g=0:k=0:z=0
1020 lprint chr$(27)+chr$(37)+chr$(57)+chr$(12):lprint
chr$(10)
1030 if z>= 192 then 1200
1040 lprint chr$(3)
1050 if k > 255 then 1080
1060 lprint chr$(-2*(point(k,z) <> 1) -4*(point(k,z+1) <> 1)
-8*(point(k,z+2) <> 1) -16*(point(k,z+3) <> 1)
-32*(point(k,z+4) <> 1) -64*(point(k,z+5) <> 1)
-128*(point(k,z+6) <> 1)) ;
1070 k=k+1 : g=g+1 : goto 1050
1080 k=0 : lprint chr$(3)+chr$(2) ; : lprint chr$(10);
1090 z=z+6 : goto 1030
1200 lprint:lprint chr$(27)+chr$(54)
```


Unfortunately the author gave me no details about how this routine should be used. I suspect it prints all non-black pixels to the printer.

Committee News

We had a committee meeting on Jan 17th where we discussed overall club policy and philosophy. Results follow ..

- 1 .. We have a number of members in the Kapiti and Hutt Valley areas who are unable to attend meetings. We will run 2 meetings a year in addition to our usual meetings in each of these areas. But we need to know just whether people will come along. Kapiti people please contact Darryl Alison 85522 Paraparaumu if you wish to attend , Hutt Valley people contact me at 896379 if you are interested. We would need to have about 10+ people along. No dates or anything yet.
- 2 .. The BUGS file ... enclosed with this newsletter is a form for a new venture in the club called The Bugs File. Any bugs in commercial software , Basic , CPM , Manuals fill this in on the form (make yourself some copies) , send it to me . The whole file will be available at meetings and will also be printed in the newsletter . No need for names on the sheet , but please use it for the benefit of all.
- 3 .. We discussed the concept of the National Committee of users group at length and have decided to support CDL's attempts to get this going . Consequently our national committee rep (ROSS) has written to other groups and CDL to try and get something started there .
- 4 .. Software written by club members. Elsewhere in here is news of our club software evaluation committee. We have decided that any member who submits a program to this committee will 1) receive a receipt , and 2) get back 3 separate lots of comments about the program. Members can submit their program to any committee member . If this is done , we guarantee absolutely that the program will not be mishandled , copied for other than evaluation purposes etc. However members who let other members outside the committee have copies of programs do so at their own risk . Any copies made by the evaluation committee will be destroyed after evaluation.
- 5 .. Starting from the meeting of February 25th we will hold a 20 minute talk on various aspects of the computers before the main meeting starts. Usually this will take the form of a BASIC tutorial and will start at 7.00 in the meeting room. Anybody interested in talking on some particular

aspect of the machine please contact me so I can schedule you in.

Program Evaluation Committee

What is it ... a 3 person committee who will evaluate your program and return comments on it to you

Why .. to provide software writers with unbiased comments on programs, particularly if you may wish to sell it later.

Who .. Darryl Alison , Andrew McMillen , Jonathan Clark

Where .. send programs on cassette PLUS DOCUMENTATION to Darryl at P.O Box 189 , Paraparaumu , or phone him with details.

and .. this committee will also provide a program assistance service. Ring Darryl if you have problems and he will put you on to the best person to try and solve it. This service will include problems with BASIC or CP/M and its components (eg DBASE2 , Wordstar , MBasic , 8080 Assembler , Pascal etc). If you want to be included as someone with particular skills who Darryl could advise members to contact please give him your name , phone number and what you can help with .

When sending programs to Darryl please include details of what the program should do , whether it is disk/cassette/either , give a name for it , and whether you intend selling it commercially later , printing it in a magazine (Bits & Bytes etc) or whatever.

